Ders 13 Genişletilmiş Notu: Heap Veri Yapısı, Heap Oluşturma ve Heap Sort

Ders 13 Genişletilmiş Notu: Heap Veri Yapısı, Heap Oluşturma ve Heap Sort

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Yeni Bir Veri Yapısı: Heap
- 2. Complete Binary Tree Hatırlatması
- 3. Bellekteki Heap Alanı ile Karıştırmamak
- 4. Heap'in Tanımı: Max Heap ve Min Heap
- 5. Heap mi, Değil mi? Üç Örnek
- 6. Heap'in Dizi ile Gösterimi
- 7. Heap'in Yüksekliği
- 8. En Büyük Elemanı Okumak
- 9. Bottom-Up Heap Oluşturma ve MAX-HEAPIFY
- 10. En Büyük Elemanı Heap'ten Almak (Extract Max)
- 11. Bir Düğümün Değerini Artırmak (Increase Key)
- 12. Heap'e Yeni Eleman Eklemek (Insert)
- 13. Top-Down Heap Oluşturma
- 14. Heap Sort
- 15. Üçlü Heap ve d'li Heap
- 16. Algoritmaların C Karşılığı

1. Yeni Bir Veri Yapısı: Heap

Hoca dönemin 7. haftasındaki bu derse yeni bir veri yapısıyla başladı. Bu veri yapısı temelde bir ağaçtır ve adı heap'tir. Stack, queue ve tree gibi bu yapıyı da İngilizce adıyla kullanmayı tercih ettiklerini söyledi. Nedeni, heap'in Türkçede birebir karşılığının olmamasıdır. Heap için bazen "yığın", bazen "yığıt" denir. Fakat aynı kelimeler stack için de kullanılabiliyor. Anlam karışıklığı olmasın diye derste stack ve heap adları ayrı tutuluyor.

Derste özellikle ikili heap (binary heap) anlatılacak. Hoca ikili olmasının çok önemli olmadığını, istenirse üçlü heap (ternary heap) ya da daha yüksek dereceli ağaçlarla başka heap'ler de kurulabileceğini söyledi. İkili heap'ten bahsediyorsak tam olarak bir ikili ağaç (binary tree) üzerinde çalışıyoruz demektir. Ama bu, her türlü özelliği olan genel bir ağaç değildir. Belirli bir özelliği taşıyan özel bir ağaçtır ve complete binary tree biçiminde düzenlenir.

2. Complete Binary Tree Hatırlatması

Hoca tahtaya bir ikili ağaç çizerek complete binary tree tanımını hatırlattı. Complete binary tree'de son seviye hariç bütün seviyelerdeki düğümler tam doludur. En alt seviye ise en soldan başlayarak sağa doğru dolar. Hocanın çizdiği ağaçta birinci, ikinci ve üçüncü düzeyler tam doluydu. Son düzeyde düğümler soldan başlayarak yerleşmişti.

Bu yapının dizi karşılığı şudur: Ağacı bir dizide tutarsak dizinin 1. gözünden k. gözüne kadar bütün elemanlar doludur. Boş gözler varsa bunlar yalnızca k'dan n'e kadar olan kısımda, yani dizinin sonunda kalır. Başka bir deyişle ağacın ilk k elemanı doludur, arada boşluk yoktur.

3. Bellekteki Heap Alanı ile Karıştırmamak

Hoca "heap" kelimesinin programlama dillerinde başka bir anlamda da kullanıldığını belirtti. Java ya da Lisp gibi dillerde çöp toplayıcının (garbage collector) yönettiği bir veri depolama alanından söz edilir. Java'daki bu alana heap space denir. Hoca bu alanı, derleyicinin bellek adreslerini tuttuğu bir bellek bölgesi olarak tarif etti.

Burada anlatılan veri yapısı ise kesinlikle böyle bir bellek alanı değildir. Bellekte yer kaplayan bir depolama alanından bahsetmiyoruz. Veri yapısı olan heap, ikili ya da üçlü ağaç biçiminde düzenlenen ve belirli bir koşulun her zaman doğru kalmasını sağlayan bir yapıdır.

> **Ders sonrası not:** Hoca Java'daki heap space'i değişkenlerin tanımlarının ve adreslerinin tutulduğu alan olarak tarif etti. Doğrusu, büyük olasılıkla şudur: Java'da heap, `new` ile oluşturulan nesnelerin tutulduğu ve çöp toplayıcının yönettiği dinamik bellek alanıdır. Yerel değişkenler ve referanslar ise genelde stack'te tutulur. Bu bellek alanının veri yapısı olan heap ile bir ilgisi yoktur.

4. Heap'in Tanımı: Max Heap ve Min Heap

4.1 Heap Ne İşe Yarar?

Heap'ler veriye hızlı erişebilmek için verinin belirli bir düzende durmasını garanti eder. Bu düzen, kökte her zaman belirli bir özelliği taşıyan elemanın durmasıdır. Bu özellik küçüklük ya da büyüklük ilişkisi olabilir. Yani heap, en büyük ya da en küçük elemanın her zaman kök düğümde olmasını garanti eden bir veri yapısıdır. Böyle bir yapıda en küçük (ya da en büyük) değeri yalnızca ilk elemanı okuyarak elde ederiz. Hangisinin kökte duracağı tamamen kurduğunuz düzene bağlıdır.

Hoca bunun bir kullanım alanı olarak sıralama problemini andı. Şimdiye kadar sıralamayı dizi üzerinde karşılaştırmalarla yaptık: kabarcık sıralaması (bubble sort), seçmeli sıralama (selection sort), eklemeli sıralama (insertion sort). Heap ile ise verinin gösterimini değiştirip o gösterimin özelliklerinden yararlanarak sıralama problemini $n \log n$ karmaşıklıkta, daha etkin çözebileceğiz.

4.2 Tanım

İkili heap'in tanımı şudur: Bir düğümdeki değer çocuklarındaki değerlerden ya her zaman büyük (ya da eşit) ya da her zaman küçüktür (ya da eşittir). İkisi aynı anda olmaz. Ağaç boyunca bu koşulun her zaman sağlanmasını garanti eden ağaç temelli veri yapısına heap denir.

- **Max heap:** En büyük eleman köktedir. Bütün i'ler için $A[\text{parent}(i)] \geq A[i]$ sağlanır.
- **Min heap:** En küçük eleman köktedir. Bütün i'ler için $A[\text{parent}(i)] \leq A[i]$ sağlanır.

Hoca burada eşitliğe dikkat çekti: İkili arama ağacında (binary search tree) eşitlik durumu yoktu, heap'te ise ebeveyn çocuğuna eşit olabilir.

Tanım özyinelemeli (recursive) olarak da verilebilir. Heap'in herhangi bir seviyesinden bir ebeveyn alıp onu kök kabul eden bir alt ağaç (alt heap, sub heap) alırsak, bu alt ağaç da kendi içinde bir heap'tir. Demek ki heap oluşturmak, kökte en büyük (ya da en küçük) elemanın durmasını garanti etmektir. Kökteki eleman kullanılıp heap'ten alınırsa, heap yeniden düzenlenerek kalan elemanların en büyüğü (ya da en küçüğü) yine köke getirilmelidir. Heap üzerindeki işlemler de aslında bunlardır.

4.3 Heap ile İkili Arama Ağacı Farkı

Hoca, ilk akla gelebilecek "bu ikili arama ağacına benziyor" düşüncesine karşı uyardı. Heap bir bakıma ona benzer ama ikili arama ağacının düzenine sahip değildir. Kökten küçüklerin solda, büyüklerin sağda duracağı gibi bir garanti yoktur. "Geçen hafta ikili arama ağacını gördük,

bütün heap işlemlerini onunla yaparım" denemez. Heap yalnızca tek bir şeyi garanti eder: Kökte en küçük değerli düğüm (min heap) ya da en büyük değerli düğüm (max heap) vardır.

> **Hocanın vurgusu:** Heap için iki koşul aynı anda sağlanmalıdır: Ağaç complete binary tree olmalıdır ve her ebeveyn çocuklarından büyük ya da eşit (max heap) veya küçük ya da eşit (min heap) olmalıdır. Bu ikisini aynı anda sağlayan ağaç heap'tir.

5. Heap mi, Değil mi? Üç Örnek

Hoca tanımdan sonra slayttaki üç ağaç üzerinde, bir ağacın heap olup olmadığını iki soru sorarak inceledi: Complete binary tree mi (CBT?), heap koşulunu sağlıyor mu (H?).

Birinci ağaç. Kökte 10 var. Dizi gösterimi `10, 5, 7, 4, 2, 1`. Dizi 1. gözden 6. göze kadar dolu, 7. göz boş.

- CBT? Evet. Bütün seviyeler tam dolu, en alt seviye soldan dolmuş, boşluk sağda.
- H? 10'un çocukları 5 ve 7, ikisinden de büyük. 5'in çocukları 4 ve 2, ikisinden de büyük. 7'nin tek çocuğu 1, ondan büyük. 4, 2 ve 1'in çocuğu olmadığı için onlara bakmaya gerek yok.
- İki koşul da sağlandığı için bu ağaç bir heap'tir.

İkinci ağaç. Dizi gösterimi `10, 5, 7, _, 2, 1`. Son seviyede soldaki bir eleman eksik.

- CBT? Hayır. Üst iki seviye tam dolu ama alt seviye soldan dolu değil, dizide arada bir boşluk var.
- Bu yüzden büyüklük küçüklük ilişkisine bakmaya bile gerek yoktur. Bu ağaç heap değildir.

Üçüncü ağaç. Dizi gösterimi `10, 5, 7, 6, 2, 1`.

- CBT? Evet. Bütün seviyeler dolu, son seviye soldan dolu.
- H? 10, çocukları 5 ve 7'den büyük. 7, çocuğu 1'den büyük. Fakat 5, çocuğu 6'dan büyük değil. Koşul bozuluyor.
- Bu ağaç da heap değildir.

Özetle ikinci ağaç complete binary tree olmadığı için, üçüncü ağaç ise büyüklük küçüklük ilişkisi sağlanmadığı için heap değildir. Yalnız birinci ağaç heap'tir.

Hoca birinci ağaç üzerinden ikili arama ağacı farkını da gösterdi. Kök 10, solundaki 5 ondan küçük, bu ikili arama ağacına uygun. Ama 7 de 10'dan küçük ve sağda duruyor. İkili arama ağacında 7'yi ararken "10'dan küçük, sola git" derdiniz. Burada 7 sağdadır. Heap'te yalnızca ebeveynin çocuklardan büyük (max heap) ya da küçük (min heap) olduğunu biliriz, o kadar.

6. Heap'in Dizi ile Gösterimi

6.1 Neden Dizi?

Heap'i tutmak için en sık kullanılan temel yapı dizidir. Bu durumda heap'ler genellikle statik boyutludur. Hoca, istenirse dinamik boyutlu ya da bağlı liste (linked list) düzeniyle teorik olarak sınırsız büyüyebilen heap'ler de gerçekleştirilebileceğini söyledi. Derste daha hızlı ve kolay olduğu için dizi gösterimi kullanılacak.

Dilden dile değişse de işlem kolaylığı için 0. göz kullanılmaz. Heap 1'den n'e kadar olan gözlerde tutulur.

Soru: 0. gözü ne yapalım o zaman? **Cevap:** Hoca, örneğin heap'teki eleman sayısının 0. gözde tutulabileceğini söyledi.

Bu durumda $A[1..n]$ dizisinin her elemanı heap ağacındaki bir düğüme karşılık gelir. Her zaman $0 \leq$ heap_size $\leq A$.length olur. Yani 10 elemanlı bir dizide heap en fazla 10, en az 0 eleman barındırır. 0 elemanlı heap boş kümedir ve bu da bir heap'tir, çünkü kök yoktur.

6.2 İndis Hesapları

Heap'in kökü, yani en büyük ya da en küçük eleman, birinci gözde, $A[1]$'de durur.

- i. düğümün sol çocuğu: $2i$

- i. düğümün sağ çocuğu: $2i + 1$
- i. düğümün ebeveyni: $\lfloor i/2 \rfloor$ (tam sayı bölmesi, alta yuvarlama)

Hoca bunu tahtada indisler üzerinde gösterdi. 1. gözün çocukları 2 ve 3'tedir. 2. gözün çocukları $2 \cdot 2 = 4$ ve $2 \cdot 2 + 1 = 5$'tedir. 3. gözün çocukları 6 ve 7'dedir. 4. gözün çocukları 8 ve 9'dadır. 5. gözün çocuğu 10'dadır; olsaydı diğer çocuğu 11. gözde olurdu. Bir düğümden ebeveynine çıkmak için indisi ikiye bölmek (tam sayı bölmesi) yeterlidir. Örneğin 5. gözden $\lfloor 5/2 \rfloor = 2$. göze çıkılır.

Böylece dizi kullanılan ikili heap'te bir düğümün çocuklarına $2i$ ve $2i + 1$ ile, ebeveynine $i/2$ ile çok hızlı erişilir.

6.3 Çocuğu Olan Düğümler

Hoca indis oklarından şu sonucu çıkardı: Çocuğu olan bütün düğümler (örnekte 1., 2., 3., 4. ve 5. düğümler) dizinin ilk $\lfloor n/2 \rfloor$ elemanında durur. Ağaç complete binary tree olduğu için $n/2$'den sonraki her şey yapraktır, yani çocuğu olmayan uç düğümdür. Bu bilgi heap oluştururken kullanılacak: Düzenleme yapılacaksa yalnızca $n/2$ elemanı kontrol etmek yeterlidir.

7. Heap'in Yüksekliği

Her ebeveynin iki çocuğu olduğu için heap ağacının yüksekliği köke kadar her seviyede ikiye dallanarak büyür. Heap'in yüksekliği kökün yüksekliğine eşittir ve n elemanlı ikili bir complete binary tree için bu $\log_2 n$ mertebesindedir. Hoca bu basitliğin heap'in hızlı çalışmasını sağladığını söyledi: Heap her düzenlendiğinde en fazla $\log n$ mertebesinde işlem yapılır.

Soru: Heap'in yüksekliği bize dizinin uzunluğunu mu veriyor? **Cevap:** Hayır. Örnekteki dizinin uzunluğu 10'dur ama heap'in yüksekliği 3'tür, yani $\log n$ mertebesindedir. Çünkü bu bir ikili ağaçtır, üstelik complete binary tree'dir. Hoca burada heap'in tanımını tekrarladı: Temelde bir dizi, ama bir ağaç olarak düşünülüyor; complete binary tree olacak ve ebeveynler çocuklardan büyük (max heap) ya da küçük (min heap) olacak.

Soru: Yüksekliği hâlâ anlamadım, $\log_2 n$ bize tam olarak neyi veriyor? **Cevap:** Hoca 3 elemanlı bir ağaç çizip öğrenciye $\log_2 3$'ü hesaplattı. Hesap makinesi 10 tabanında hesapladığı için hoca "logaritma 2'ye böl" dedi: $\log_2 3 = \log_{10} 3 / \log_{10} 2 \approx 1.58$. Sonra 4 elemanlı ağaca geçti. Bu ağacın yüksekliği 2'dir ve $\log_2 4 = 2$'dir. 7 elemana kadar yükseklik "2 nokta bir şey" ile devam eder, 8 elemana gelince $\log_2 8 = 3$ olur ve bir seviye daha eklenir. Yani n elemanlı complete ikili ağacın yüksekliği $\log_2 n$ mertebesindedir.

> **Ders sonrası not:** Hoca 3 elemanlı ağaç için "1.58'i yuvarlayınca 2 değil mi, en fazla yükseklik 2 olabiliyor" dedi, ardından "aslında 1" diye düzeltir gibi oldu. Doğrusu: n elemanlı heap'in yüksekliği $\lfloor \log_2 n \rfloor$'dir. 3 elemanlı ağacın yüksekliği $\lfloor 1.58 \rfloor = 1$'dir. 4 ile 7 eleman arasında yükseklik 2, 8 ile 15 eleman arasında 3'tür. 10 elemanlı örnekte $\lfloor \log_2 10 \rfloor = \lfloor 3.32 \rfloor = 3$ olur; hocanın söylediği 3 doğrudur.

Bu yüzden heap ağacının yüksekliği $\log n$'i geçemez. Bir düzenleme işleminde "bununla karşılaştır, şununla karşılaştır" diye ilerlerken yapılabilecek en fazla karşılaştırma sayısı da $\log n$ mertebesindedir. "Acaba şu tarafa da mı baksam, onun çocuklarına da mı baksam" diye düşünmeye gerek yoktur. Ağacın yüksekliği boyunca yürümek yeterlidir.

8. En Büyük Elemanı Okumak

İlk işlem en büyük (ya da en küçük) elemanı bulmaktır. Heap tek bir şeyi garanti ettiği için, yani en büyük eleman kökte durduğu için, yalnızca kökü sorgulamak yeterlidir:

```
HEAP-MAXIMUM(A)
  return A[1]
```

Yapılan tek işlem birinci gözdeki değeri döndürmektir. Bu sabit zamanlı bir işlemdir, karmaşıklığı $O(1)$'dir. Hoca bunun güzelliğini vurguladı: Ne kadar veri olursa olsun, onu heap düzeninde tutarsanız "en büyük değer neydi?" sorusunun cevabını $O(1)$'de alırsınız.

9. Bottom-Up Heap Oluşturma ve MAX-HEAPIFY

9.1 İki Yaklaşım

Elimizde bir dizi varsa onunla heap oluşturmanın iki yolu vardır: aşağıdan yukarı yaklaşım (bottom-up approach) ve yukarıdan aşağı yaklaşım (top-down approach). Önce bottom-up yaklaşım anlatıldı. Top-down yaklaşım insert işleminden sonra (Bölüm 13) geldi.

9.2 Algoritma

Bottom-up yöntemin adımları:

- **Adım 0:** Verilen eleman grubunu soldan başlayarak sırayla bir diziye yaz. 1. gözden n. göze kadar dolan bu dizi bir complete binary tree oluşturur.
- **Adım 1:** Bundan sonra her ebeveynin çocuklarından büyük olmasını garanti etmek gerekir. Bunun için yalnızca çocuğu olan düğümler kontrol edilir. Bir düğüm çocuklarından küçükse çocuğuyla yer değiştirir. Aşağı inen değerin de çocukları varsa onlarla tekrar karşılaştırılır. Yani küçük değer özyinelemeli olarak ağacın alt seviyelerine doğru itilir.

Çocuğu olan düğümler ilk $\lfloor n/2 \rfloor$ gözde durduğu için döngü $n/2$'den 1'e doğru kurulur:

```
BUILD-MAX-HEAP(A, n)
  for i <- n/2 downto 1
    MAX-HEAPIFY(A, i, n)
```

`MAX-HEAPIFY` fonksiyonuna dizi, o anda bakılan göz i ve heap'teki toplam eleman sayısı n verilir. Fonksiyon düğümü sol çocuğu ($2i$) ve sağ çocuğu ($2i+1$) ile karşılaştırır. Bu üçlünün en büyüğü (largest) bulunur ve en büyük olanın ebeveyn olması gerekir. Largest düğümün kendisi değilse, çocuklardan biri largest demektir; o zaman ikisi yer değiştirir. Yer değiştirmeden sonra aşağı inen değerin bir alt seviyede yeniden kontrol edilmesi gerekip gerekmediğine bakmak için fonksiyon kendini largest gözü için çağırır.

```
MAX-HEAPIFY(A, i, n)
  l <- LEFT(i)
  r <- RIGHT(i)
  if l <= n and A[l] > A[i] then largest <- l
  else largest <- i
  if r <= n and A[r] > A[largest] then largest <- r
  if largest != i then
    exchange A[i] <-> A[largest]
    MAX-HEAPIFY(A, largest, n)
```

> **Ders sonrası not:** Kısa nota göre slayttaki `MAX-HEAPIFY` kodunda koşul büyük olasılıkla `if largest = i` biçiminde yazılmıştı. Doğrusu yukarıdaki gibi `if largest != i` olmalıdır. Hoca da anlatırken "largest kendisi değilse yer değiştir" dedi.

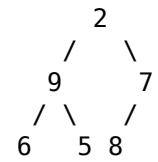
Hoca karmaşıklığı şöyle açıkladı: `MAX-HEAPIFY` $n/2$ kez çağrılıyor. Büyük O gösteriminde $\frac{1}{2}n$'deki $\frac{1}{2}$ katsayısı önemli değildir, bu yüzden `BUILD-MAX-HEAP` n mertebesinde, yani $O(n)$ zaman karmaşıklığına sahiptir.

> **Ders sonrası not:** $O(n)$ sonucu doğrudur ama "fonksiyon $n/2$ kez çağrılıyor" gerekçesi tek başına yetmez. Her `MAX-HEAPIFY` çağrısı en kötü durumda $O(\log n)$ sürdüğü için kaba hesap $O(n \log n)$ verir. Sonucun $O(n)$ çıkmasının nedeni, düğümlerin çoğunun alt seviyelerde olması ve bunların ancak birkaç seviye inebilmesidir. Sıkı analiz $\sum_{h=0}^{\lfloor \lg n \rfloor} \frac{n}{2^{h+1}} O(h) = O(n)$ verir.

9.3 Örnek 1: `2, 9, 7, 6, 5, 8`

İlk 10 dakikalık aradan sonra hoca algoritmayı bir örnek üzerinde işletti.

Adım 0: Dizi `2, 9, 7, 6, 5, 8` olarak 1. ile 6. gözlere yazılır. Ağaç görünümünde kök 2'dir; 9 ve 7 onun çocuklarıdır; 6 ve 5, 9'un çocuklarıdır; 8, 7'nin sol çocuğudur.



$n = 6$ olduğu için çocuğu olan son düğüm $6/2 = 3$. gözdür. Sırasıyla 3., 2. ve 1. gözlere bakılır.

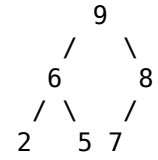
i = 3 (değer 7): Sol çocuk 8, sağ çocuk yok. Largest önce 7 kabul edilir; 8 > 7 olduğu için largest 8 olur. Sağ çocuk olmadığından başka karşılaştırma yapılmaz. 7 ile 8 yer değiştirir. 7'nin yeni yeri 6. göz; orada çocuk yok, özyinelemeli çağrı bir şey yapmaz. 3. indisteki alt ağaç artık kendi içinde bir heap'tir. Dizi: `2, 9, 8, 6, 5, 7`.

i = 2 (değer 9): Sol çocuk 6, sağ çocuk 5. Hiçbiri 9'dan büyük değil, largest kendisidir. Yer değiştirme ve alt seviyeye inme gerekmez. Dizi değişmez: `2, 9, 8, 6, 5, 7`.

i = 1 (değer 2): Kök düğüm. Largest önce 2. Sol çocuk 9 daha büyük, largest 9. Sağ çocuk 8, 9'dan büyük değil, largest hâlâ 9. 2 ile 9 yer değiştirir. Bu yetmez: 2 indiği 2. gözde yeniden heapify edilmelidir. Orada largest önce 2; sol çocuk 6 daha büyük, largest 6; sağ çocuk 5 daha büyük değil. 2 ile 6 yer değiştirir. 2 artık yaprak olduğu için işlem biter.

Adım	İşlem	Dizi
Başlangıç		2, 9, 7, 6, 5, 8
i = 3	7 ↔ 8	2, 9, 8, 6, 5, 7
i = 2	değişiklik yok	2, 9, 8, 6, 5, 7
i = 1	2 ↔ 9, sonra 2 ↔ 6	9, 6, 8, 2, 5, 7

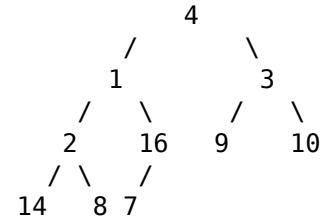
Sonuç: `9, 6, 8, 2, 5, 7`.



Hoca sonucu şöyle yorumladı: Verilen girdi dizisi max heap koşulunu sağlayan bir düzene yeniden getirildi. Birinci gözde dizinin en büyük elemanı, 9, var. 9, çocukları 6 ve 8'den büyüktür. 6, çocukları 2 ve 5'ten büyüktür. 8, çocuğu 7'den büyüktür.

9.4 Örnek 2 (Kitap Örneği): `4, 1, 3, 2, 16, 9, 10, 14, 8, 7`

Daha kalabalık bir örnek. Dizi 10 elemanlıdır.



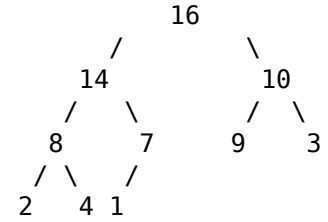
Çocuğu olan en son düğüm $10/2 = 5$. gözdedir (değer 16). Döngü 5, 4, 3, 2, 1 sırasıyla ilerler.

- **i = 5 (16):** Tek çocuğu 7. Largest 16, kendisine eşit. Alt ağaç zaten heap'tir.
- **i = 4 (2):** Largest önce 2. Sol çocuk 14 daha büyük, largest 14. Sağ çocuk 8, 14'ten küçük. 14 ile 2 yer değiştirir. Artık 4. ve 5. düğümlerin alt ağaçları heap'tir.
- **i = 3 (3):** Largest önce 3. Sol çocuk 9 daha büyük, largest 9. Sağ çocuk 10, 9'dan da büyük, largest 10. 10 yukarı çıkar, 3 aşağı iner.
- **i = 2 (1):** Largest önce 1. Sol çocuk 14 daha büyük, largest 14. Sağ çocuk 16 daha da büyük, largest 16. 16 yukarı, 1 aşağı (5. göze) iner. 1'in orada çocuğu (7) olduğu için kontrol sürer: 7 > 1, 7 yukarı çıkar, 1 aşağı iner.

- **i = 1 (4):** Largest önce 4. Sol çocuk 16, largest 16. Sağ çocuk 10, 16'dan küçük. 16 köke çıkar, 4 aşağı iner. 4'ün çocukları 14 ve 7: 14 yukarı, 4 aşağı. 4'ün çocukları 2 ve 8: 8 yukarı, 4 aşağı.

Adım	İşlem	Dizi
Başlangıç		4, 1, 3, 2, 16, 9, 10, 14, 8, 7
i = 5	değişiklik yok	4, 1, 3, 2, 16, 9, 10, 14, 8, 7
i = 4	2 ↔ 14	4, 1, 3, 14, 16, 9, 10, 2, 8, 7
i = 3	3 ↔ 10	4, 1, 10, 14, 16, 9, 3, 2, 8, 7
i = 2	1 ↔ 16, 1 ↔ 7	4, 16, 10, 14, 7, 9, 3, 2, 8, 1
i = 1	4 ↔ 16, 4 ↔ 14, 4 ↔ 8	16, 14, 10, 8, 7, 9, 3, 2, 4, 1

Sonuç: `16, 14, 10, 8, 7, 9, 3, 2, 4, 1`.



Max heap olduğu için en büyük eleman köktedir; $A[1]$ sorgulanınca en büyük değer okunur.

Hoca yöntemin neden "bottom-up" adını aldığını özetledi: $n/2$'den, yani çocuğu olan en son ebeveynden başlanır. Önce onun heapify işlemi bitirilir, sonra bir öncekine, sonra bir öncekine geçilir (5, 4, 3, 2, 1). En dipten köke doğru düzen sağlandığı ve gerektiğinde özyinelemeli olarak alt çocuklar da kontrol edildiği için yönteme bottom-up denir.

9.5 Min Heap İçin

Hoca hep max heap örneği verdiğini, min heap için ayrı örneğe gerek olmadığını söyledi. Kontroller tersten yapılır: "büyük mü" yerine "küçük mü" diye bakılır, largest yerine smallest bulunur ve en küçük değer köke taşınır. Ebeveynden büyük olanlar çocuk olarak aşağı itilir. Tek fark karşılaştırma işaretlerini ters çevirmektir.

10. En Büyük Elemanı Heap'ten Almak (Extract Max)

10.1 Algoritma

En büyük elemanı $O(1)$'de okuyabiliyoruz. Ama onu kullanabilmek için heap'ten çekip almamız da gerekir. Adımlar şunlardır:

- Heap boş olmamalıdır. Heap boşsa ($n < 1$) eleman alınamaz; hiçbir şey yapmadan ya da "heap'te eleman yok" gibi bir hata vererek durulur.
- Kökteki değerin bir kopyası alınır: `max <- A[1]`. Çünkü bu değer döndürülecektir ve kök birazdan üzerine yazılacaktır.
- Heap bilerek ve isteyerek bozulur: En son eleman köke yazılır, `A[1] <- A[n]`.
- Heap boyutu bir azaltılır; o eleman artık heap'in içinde değildir.
- Kalan $n - 1$ eleman için kökten `MAX-HEAPIFY` çağrılır.
- `max` döndürülür.

```
HEAP-EXTRACT-MAX(A, n)
  if n < 1 then
    error "heap'te eleman yok"
  max <- A[1]
  A[1] <- A[n]
  n <- n - 1
  MAX-HEAPIFY(A, 1, n)
  return max
```

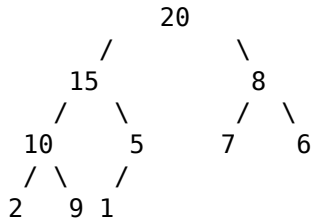
Hoca slayttaki parametre sırasını ve boş heap kontrolünün eksik kısmını anlatırken tamamladı ("eksikleri de on the fly tamamlıyoruz").

Karmaşıklık: Kopyalama ve atama sabit işlemdir. `MAX-HEAPIFY` ise $n-1$ elemanlı ağacın yüksekliği boyunca en fazla $\log(n-1)$ seviye iner. Bu yüzden extract max işlemi en fazla $O(\log n)$'dir. Hoca bunun nedeninin dersin başında söylendiği gibi ağacın complete binary tree olması olduğunu hatırlattı.

> **Ders sonrası not:** Hoca son elemanın köke alınma nedenini "o zaten olabilecek en küçük çocuktur, en küçük çocuklardan biridir" diye açıkladı. Son eleman bir yapraktır ama heap'in en küçük elemanı olmak zorunda değildir. Köke alınmasının asıl nedeni, diziden çıkarılabilecek tek gözün son göz olması ve böylece complete binary tree yapısının korunmasıdır. Köke hangi değer gelirse gelsin `MAX-HEAPIFY` onu doğru yerine indirir.

10.2 Örnek: `20, 15, 8, 10, 5, 7, 6, 2, 9, 1`

Hoca slayttaki 10 elemanlı bir max heap ile çalıştı. Hem complete binary tree'dir hem de her ebeveyn çocuklarından büyüktür.



Birinci extract max: 20 kopyalanır. Son eleman 1 köke yazılır, heap boyutu 10'dan 9'a iner. Artık max heap değildir, çünkü köke en alt seviyeden bir değer yazıldı. Kökten `MAX-HEAPIFY` çağrılır:

- 1; çocukları 15 ve 8. Largest 15. 15 yukarı, 1 aşağı.
- 1; çocukları 10 ve 5. 10 yukarı, 1 aşağı.
- 1; çocukları 2 ve 9. 9 yukarı, 1 aşağı. 1 kendi seviyesini buldu.

Kökte artık 15 var. Dizi: `15, 10, 8, 9, 5, 7, 6, 2, 1`.

> **Ders sonrası not:** Hoca bu adımın sonucunu "15, 18, 9, 5, 7, 6, 2, 1" diye okudu. Python ile doğrulanan sonuç `15, 10, 8, 9, 5, 7, 6, 2, 1`'dir.

Hoca $n = 9$ için heapify'ın 4. düğümden başladığını da söyledi ($9/2 = 4$). Ancak extract max'ta yalnızca kök bozulduğu için `MAX-HEAPIFY` doğrudan 1. gözden çağrılır.

İkinci extract max: 15 kopyalanır. Son eleman 1 yine köke yazılır, boyut 8 olur.

- 1; çocukları 10 ve 8. 10 yukarı, 1 aşağı.
- 1; çocukları 9 ve 5. 9 yukarı, 1 aşağı.
- 1; tek çocuğu 2. 2 yukarı, 1 aşağı. 1 yine en alt seviyeye indi.

Kökte artık 10 var. Dizi: `10, 9, 8, 2, 5, 7, 6, 1`.

Adım	Alınan	Heap
Başlangıç		20, 15, 8, 10, 5, 7, 6, 2, 9, 1
1. extract max	20	15, 10, 8, 9, 5, 7, 6, 2, 1
2. extract max	15	10, 9, 8, 2, 5, 7, 6, 1

Hoca devam edilirse sırayla 10, 9, 8 gibi değerlerin geleceğini söyledi. Kalan elemanların en büyüğü her seferinde birinci gözde kalır ve her extract max en fazla $\log n$ işlemle yapılır.

11. Bir Düğümün Değerini Artırmak (Increase Key)

11.1 Algoritma

Heap'te aşağılarda duran bir düğümün değerini artırırsak, onun aslında daha yukarıda durması gerektiğini söylemiş oluruz. Değer artınca ebeveyninden küçük kalma koşulu bozulabilir. Bu koşul kontrol edilmeli ve düğüm gerekirse yukarı taşınmalıdır.

- i. göze yeni bir anahtar değeri (key) atanacaktır. Bu değer mevcut değerden büyük ya da eşit olmalıdır. Küçükse hiçbir şey yapılmadan durulur. Hoca, değer azaltılırsa yapının zaten değişmediğini söyledi.
- `A[i] <- key` atanır. Bu atama heap'i bilerek bozabilir.
- $i > 1$ olduğu ve ebeveyn bu değerden küçük olduğu sürece düğüm yanlış yerdedir. Ebeveyniyle yer değiştirir ve i ikiye bölünerek bir üst seviyeye çıkılır.

```
HEAP-INCREASE-KEY(A, i, key)
  if key < A[i] then
    return
  A[i] <- key
  while i > 1 and A[PARENT(i)] < A[i]
    exchange A[i] <-> A[PARENT(i)]
    i <- PARENT(i)
```

En kötü durum, en alt seviyedeki bir değerin yeni maksimum olacak şekilde artırılmasıdır. Bu durumda değer köke kadar $\log n$ kez yukarı taşınır. Karmaşıklık $O(\log n)$'dir.

> **Hocanın vurgusu:** Heap ağacının yüksekliği $\log n$ olduğu için heap üzerinde tanımlanan ekleme, silme ve düzeltme işlemlerinin hepsi en kötü durumda ağacın yüksekliği kadar, yani $O(\log n)$ karmaşıklığa sahiptir.

Hoca slayttaki bir animasyonu da gösterdi: En alt seviyedeki bir düğümün değeri 22 yapıldı. 22 önce 10 ile karşılaştırılıp yukarı alındı, sonra 15 ile, sonra 20 ile karşılaştırılıp köke çıktı. 10 elemanlı heap'te seviye sayısı 3 olduğu için 3 karşılaştırma ve yer değiştirme yapıldı. Yani 10 işlem yerine 3 işlemle, en kötü durumda bile en alt seviyedeki bir değer en üste taşınabildi.

11.2 Örnek (Kitaptan): 4 Değerini 15 Yapmak

Bölüm 9.4'te oluşturulan `16, 14, 10, 8, 7, 9, 3, 2, 4, 1` heap'inde 9. gözdeki 4 değeri 15 yapılır.

- 4 silinip yerine 15 yazılır. Heap bilerek bozuldu. Heap'in maksimumu 16 olduğu için 15'in köke kadar çıkmayacağı bilinir; ama 16 ile de karşılaştırılacağı için en kötü durumda yine $\log n$ mertebesinde karşılaştırma yapılır.
- 15 ebeveyni 8 ile karşılaştırılır. 15 daha büyük, yanlış yerde. 15 yukarı, 8 aşağı.
- 15 yeni ebeveyni 14 ile karşılaştırılır. 15 daha büyük. 15 yukarı, 14 aşağı.
- 15 ebeveyni 16 ile karşılaştırılır. 15 daha küçük, daha fazla yukarı çıkamaz ve durur. Burada yer değiştirme yapılmaz.

Adım	İşlem	Dizi
Başlangıç		16, 14, 10, 8, 7, 9, 3, 2, 4, 1
4 → 15	atama	16, 14, 10, 8, 7, 9, 3, 2, 15, 1
1. karşılaştırma	15 ↔ 8	16, 14, 10, 15, 7, 9, 3, 2, 8, 1
2. karşılaştırma	15 ↔ 14	16, 15, 10, 14, 7, 9, 3, 2, 8, 1
3. karşılaştırma	15 < 16, dur	16, 15, 10, 14, 7, 9, 3, 2, 8, 1

Toplam 3 karşılaştırma ve 2 yer değiştirme yapıldı. İşlem yine $\log n$ mertebesindedir.

Hoca burada iki önemli gözlem yaptı:

- **Aşağı inen değeri tekrar kontrol etmeye gerek yoktur.** 14 aşağı indiğinde onu çocuklarıyla (örneğin 7 ile) karşılaştırmak gerekmez. 14 zaten eski ebeveyndi, yani o alt ağacın eski largest'ıydı. 14 eskiden bu düğümlerin bir iki kademe üzerindeydi, ebeveynin ebeveyniydi, yani atasıydı. Heap düzeni yüzünden onlardan büyük olmak zorundadır.

Eski largest'tan daha büyük bir değeri yukarı taşıyorsanız alt düzene yeniden bakmanız gerekmez.
- **Yukarı çıkan değer durduğunda iş biter.** 15, 16'dan küçük olduğu için orada durdu; ekstra yer değiştirme olmadı.

12. Heap'e Yeni Eleman Eklemek (Insert)

12.1 Derste Tartışma

Hoca, heap düzenini sağlayan bir ağaca yeni bir elemanın nasıl ekleneceğini sınıfa sordu. Önce cevap gelmedi. Sonra bir öğrenci yapraktaki bir elemanı silip yerine yeni elemanı eklemeyi önerdi. Hoca "ben sana eleman sil demedim, yeni eleman ekle dedim" diyerek bunu reddetti. Öğrenci bu kez "sona ekleyip heapify yapacağız" dedi. Hoca sona eklemenin aslında heap yapısını bozduğunu belirtti ve kendi yöntemini gösterdi.

12.2 Eksi Sonsuz Hilesi

Yeni eleman ekleme sırasında önce heap düzeni bozulmamalıdır. Max heap'te sona çok küçük bir değer, yani minimum değer eklenirse heap bozulmaz. Algoritmada bu değer $-\infty$ ile gösterilir. Hoca, uygulamada bunun dilin sınır değerleri (örneğin en küçük tam sayı sabiti) ile yapılabileceğini söyledi. $-\infty$ eklenince heap'e yeni bir eleman eklenmiş olur ama düzen değişmez. Sonra bu $-\infty$ değeri, eklenmek istenen değerle `HEAP-INCREASE-KEY` çağrılarak artırılır. Değer gerekiyorsa yukarı çıkar.

```
MAX-HEAP-INSERT(A, n, key)
  n <- n + 1
  A[n] <- -infinity
  HEAP-INCREASE-KEY(A, n, key)
```

Karmaşıklık $O(\log n)$'dir. En kötü durumda yeni değer en alttan köke kadar çıkar.

12.3 Örnek: 12 ve 20 Eklemek

Başlangıç heap'i Bölüm 11.2'nin sonucu: `16, 15, 10, 14, 7, 9, 3, 2, 8, 1`.

12 ekleme: 11. göze $-\infty$ yazılır. Bu göz 7'nin (5. göz) sağ çocuğudur. Increase key ile değer 12 yapılır.

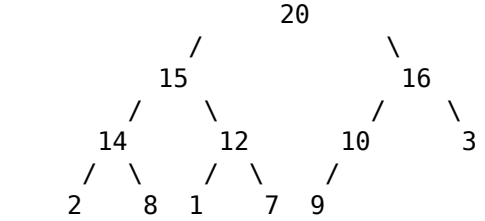
- 12 ile 7 karşılaştırılır. 12 yukarı, 7 aşağı.
- 12 ile 15 karşılaştırılır. 12 daha fazla yukarı çıkamaz, durur.

Dizi: `16, 15, 10, 14, 12, 9, 3, 2, 8, 1, 7`.

20 ekleme: 12. göze $-\infty$ yazılır. Bu göz 9'un (6. göz) çocuğudur. Increase key ile değer 20 yapılır.

- 20 ile 9 karşılaştırılır. 9 aşağı, 20 yukarı.
- 20 ile 10 karşılaştırılır. 10 aşağı, 20 yukarı.
- 20 ile 16 karşılaştırılır. 20 yukarı, 16 aşağı. 20 köke ulaştı.

Dizi: `20, 15, 16, 14, 12, 10, 3, 2, 8, 1, 7, 9`.



Hoca 20'nin dördüncü seviyeden üçe, ikiye ve bire, yani köke kadar çıktığını belirtti. Yükseklik kadar karşılaştırma ve taşıma yapıldı. $-\infty$ eklemek hiçbir şeyi değiştirmedi; increase key değeri olması gereken yere çıkardı.

13. Top-Down Heap Oluşturma

13.1 Yöntem

Bottom-up yöntemde en dipteki ebeveynden başlanıyor, onun alt ağacının heap olması garanti ediliyor ve köke doğru ilerleniyordu (`for i <- n/2 downto 1`). Kök de düzeltildiğinde bütün ağaç heap olmuş oluyordu.

Hoca, insert işlemini kullanarak alternatif bir heap oluşturma yolu olup olmadığını sordu ve "evet" dedi. Bu yöntem yukarıdan aşağı ilerler (top-down approach):

- Boş heap ile başlanır.
- İlk eleman eklenir. Tek elemanlı ağaç zaten heap'tir.
- 2'den n'e kadar her eleman sırayla insert edilir. Gerekiyorsa yukarı taşınır.

```
BUILD-MAX-HEAP-TOPDOWN(A, n)
  heap_size <- 1
  for i <- 2 to n
    MAX-HEAP-INSERT(A, heap_size, A[i])
```

13.2 İki Yöntem Aynı Heap'i Verir mi?

Hoca iki yöntemle oluşturulan heap'lerin aynı olup olmayacağını sordu. Bir öğrenci "olmaz" dedi. Hoca bunu `1, 2, 3` girdisiyle denedi.

Bottom-up: Dizi `1, 2, 3` olarak yazılır. $\lfloor 3/2 \rfloor = 1$ olduğu için yalnızca 1. göze bakılır. Largest önce 1; sol çocuk 2 daha büyük, largest 2; sağ çocuk 3 daha büyük, largest 3. Largest kendisi değil, 3 ile 1 yer değiştirir. Sonuç: `3, 2, 1`.

Top-down:

- 1 eklenir: 1.
- 2 eklenir, 1'in çocuğu olur. 2 > 1, yukarı taşınır: `2, 1`.
- 3 için sona $-\infty$ eklenir, düzen bozulmaz. Increase key ile 3 yapılır. 3 ile 2 karşılaştırılır, 3 yukarı çıkar, 2 onun yerine iner: `3, 1, 2`.

Yöntem	Sonuç
Bottom-up	3, 2, 1
Top-down	3, 1, 2

İki dizi eşit değildir. Ama ikisinde de en büyük eleman köktedir ve her ebeveyn çocuklarından büyüktür.

> **Hocanın vurgusu:** Bottom-up ve top-down yaklaşımlarla üretilen heap'ler aynı heap olmak zorunda değildir. Sağlanan şey yalnızca iki koşuldur: complete binary tree olmak ve her ebeveynin çocuklarından büyük (max heap) ya da küçük (min heap) olması.
>
> **Ek bilgi (derste anlatılmadı):** İki yöntemin karmaşıklığı da farklıdır. Top-down yöntemde n kez $O(\log n)$'lik insert yapıldığı için en kötü durum büyük olasılıkla $O(n \log n)$'dir. Bottom-up yöntem ise Bölüm 9.2'deki notta belirtildiği gibi $O(n)$'dir.

14. Heap Sort

14.1 Fikir

İkinci aradan sonra hoca heap'in tanımını kısaca tekrarladı. Heap bir complete binary tree'dir ve tek bir şeyi sağlar: Ebeveynler çocuklardan hep küçük ya da hep büyüktür; bu, özyinelemeli olarak bütün ebeveynler için geçerlidir. Bu yapıya heap, heap düzeni ya da heap ağacı (heap tree) denir; hepsi aynı anlama çıkar.

Heap'in en temel özelliği, en büyük ya da en küçük elemanı bulmak için özel bir çaba gerekmemesidir; birinci göz okunur. Hoca bu özelliğin bir problemi çözmekte kullanılıp kullanılamayacağını sordu. Cevap evettir: Sıralama problemi heap veri yapısıyla çözülerek toplam işlem karmaşıklığı azaltılabilir. Heap işlemleriyle sıralama yapıldığı için yöntemin adı heap sort'tur.

14.2 Algoritma

Girdi, n elemanlı bir $A[1..n]$ dizisidir.

- Önce `BUILD-MAX-HEAP` çağrılır. Artık elimizde bir heap var ve en büyük elemanın kökte olduğunu biliyoruz.
- Birinci gözdeki en büyük eleman, heap'in son elemanıyla bilerek yer değiştirilir. Küçükten büyüğe sıralamada en büyük eleman en sonda yer alır; bu sağlanmış olur.
- Heap boyutu bir azaltılır. Bu, en büyük elemanı oyun dışında bırakmaktır: "Sen artık heap'in içinde değilsin, yerini buldun, n. pozisyona yerleş."
- Yeni kök bozuktur, bilerek bozuldu. Kökten `MAX-HEAPIFY` çağrılır.
- Tek eleman kalana kadar bu işlem tekrarlanır: kökü sona at, heap boyutunu azalt, heapify et.

```
HEAPSORT(A, n)
  BUILD-MAX-HEAP(A, n)
  for i <- n downto 2
    exchange A[1] <-> A[i]
    n <- n - 1
    MAX-HEAPIFY(A, 1, n)
```

14.3 Karmaşıklık

Hoca karmaşıklığı adım adım topladı:

- `BUILD-MAX-HEAP`: $O(n)$.
- Her heapify en kötü durumda $O(\log n)$.
- n. elemandan 2. elemana kadar, tek eleman kalana kadar $n-1$ kez bir $O(1)$ yer değiştirme ve bir $O(\log n)$ heapify yapılır.

Toplam: $O(n) + (n-1) \cdot O(\log n)$. Hoca önce "$n + n \log n$" yazdı, sonra "$n - 1$ log n deseydik daha doğru olacaktı" diye düzeltti. Büyük O gösteriminde sabit katsayılı düşük terim atılır, sonuç $O(n \log n)$'dir.

14.4 Örnek 1: `16, 14, 10, 8, 7, 9, 3, 2, 4, 1`

Hoca kitaptaki örnekte oluşturulan heap'i kullandı. Bu dizi aslında girdi olarak verilmemişti; Bölüm 9.4'te `4, 1, 3, 2, 16, 9, 10, 14, 8, 7` girdisi heapify edilerek bu hale gelmişti. $n = 10$.

n = 10 → 9: 16 ile son eleman 1 yer değiştirir. 16 dizinin son gözüne yazılır; slaytta koyu renkle gösterilir ve ağaçla bağı koparılır. Heap boyutu 9 olur. Kökteki 1 bilerek bozuldu, heapify edilir: 1, 14, 10 → 14 yukarı, 1 aşağı; 1, 8, 7 → 8 yukarı, 1 aşağı; 1, 2, 4 → 4 yukarı, 1 aşağı. Heap: `14, 8, 10, 4, 7, 9, 3, 2, 1`.

n = 9 → 8: Son eleman (9. göz) 1 ile kök 14 yer değiştirir. 14 aşağı gelir, bağı koparılır. Heapify: 1, 10, 8 → 10 yukarı (bu sefer 1 sağ tarafa gider); 1, 9, 3 → 9 yukarı. Heap: `10, 8, 9, 4, 7, 1, 3, 2`. Dizinin son iki elemanı 14 ve 16'dır.

n = 8 → 7: 10 ile 2 yer değiştirir, 10'un bağı koparılır. Heapify: 2, 8, 9 → 9 yukarı; 2, 3, 1 → 3 yukarı. Heap: `9, 8, 3, 4, 7, 1, 2`.

n = 7 → 6: 9 ile 2 yer değiştirir. Heapify: 2, 8, 3 → 8 yukarı; 2, 4, 7 → 7 yukarı. Heap: `8, 7, 3, 4, 2, 1`.

n = 6 → 5: 8 ile 1 yer değiştirir. Heapify: 1, 7, 3 → 7 yukarı; 1, 4, 2 → 4 yukarı. Heap: `7, 4, 3, 1, 2`.

n = 5 → 4: 7 ile 2 yer değiştirir. Heapify: 2, 4, 3 → 4 yukarı; 2'nin altında 1 var, değişiklik yok. Heap: `4, 2, 3, 1`.

n = 4 → 3: 4 ile 1 yer değiştirir. Heapify: 1, 2, 3 → 3 yukarı, 1 aşağı. Heap: `3, 2, 1`.

n = 3 → 2: 3 ile 1 yer değiştirir. Heapify: 1 yukarıda, 2 aşağıda; 2 daha büyük, 2 yukarı çıkar. Heap: `2, 1`.

n = 2 → 1: 2 ile 1 yer değiştirir. Tek eleman kaldı; tek eleman kendi içinde zaten sıralıdır. İşlem biter.

Heap boyutu	Heap kısmı	Sıralı kısım
10	16, 14, 10, 8, 7, 9, 3, 2, 4, 1	
9	14, 8, 10, 4, 7, 9, 3, 2, 1	16
8	10, 8, 9, 4, 7, 1, 3, 2	14, 16
7	9, 8, 3, 4, 7, 1, 2	10, 14, 16
6	8, 7, 3, 4, 2, 1	9, 10, 14, 16
5	7, 4, 3, 1, 2	8, 9, 10, 14, 16
4	4, 2, 3, 1	7, 8, 9, 10, 14, 16
3	3, 2, 1	4, 7, 8, 9, 10, 14, 16
2	2, 1	3, 4, 7, 8, 9, 10, 14, 16
1	1	2, 3, 4, 7, 8, 9, 10, 14, 16

Çıktı: `1, 2, 3, 4, 7, 8, 9, 10, 14, 16`.

Hoca şunu vurguladı: Sona doğru, n, $n-1$, $n-2$, … konumlarına taşınan elemanlar en büyükten en küçüğe doğru yerlerini buldu. Her heapify en kötü durumda $\log n$ işlemdir ama heap küçüldükçe işlem sayısının azalarak gittiği görülür.

14.5 Dönüştür ve Fethet (Transform and Conquer)

Hoca heap sort'un algoritmik yaklaşımını şöyle yorumladı: Dizi olarak verilen girdi, heap düzenine dönüştürülerek farklı bir biçimde temsil edildi. Sonra bu dönüştürülmüş temsilin özelliği (en büyük elemanın hep kökte olması) kullanılarak problem çözüldü. Problem uzayı, yani veri yapısı değiştirilerek (transform) çözüm kolaylaştırıldı ve problem fethedildi (conquer).

> **Hocanın vurgusu:** Heap sort, "dönüştür ve fethet" (transform and conquer) yaklaşımının en güzel ve en temiz örneklerinden biridir. Veri heap olarak düzenlenir, heap'in özelliği sürekli kullanılır (exploit edilir): max eleman tekrar tekrar çıkarılır, heap boyutu azaltılır ve heap yeniden kurulur.

Hocaya göre burada özel bir karşılaştırma stratejisi yoktur. Her adımda elemanı sona taşı, yer değiştir, kalanları heapify et. Dönüştürülmüş girdi üzerinde adeta "körlemesine", azalan sayıda işlemle sonuca ilerlenir.

14.6 Örnek 2: `2, 9, 7, 6, 5, 8`

Hoca Bölüm 9.3'teki diziyle ikinci bir örnek yaptı. Önce bottom-up heapify yeniden gösterildi: 3. eleman 7, çocuğu 8 ile yer değiştirir; 9'da değişiklik olmaz; kök 2 önce 9 ile, sonra 6 ile yer değiştirir. Heap: `9, 6, 8, 2, 5, 7`.

Ardından ikinci aşamada kök ile son eleman sürekli yer değiştirilir:

- **9 ↔ 7:** 9 yerini buldu; heap boyutu 5. Heap kısmı `7, 6, 8, 2, 5`. Heapify: 8 yukarı, 7 aşağı. Heap: `8, 6, 7, 2, 5`.
- **8 ↔ 5:** Boyut 4. Heap kısmı `5, 6, 7, 2`. Heapify: 7 yukarı, 5 aşağı. Heap: `7, 6, 5, 2`.
- **7 ↔ 2:** Boyut 3. Heap kısmı `2, 6, 5`. Heapify: 6 ile 2 yer değiştirir. Heap: `6, 2, 5`.
- **6 ↔ 5:** Boyut 2. Heap kısmı `5, 2`. 5, 2'den büyük olduğu için heapify bir şey değiştirmez.
- **5 ↔ 2:** Boyut 1. Tek eleman kaldı, kendi içinde sıralıdır.

Heap boyutu	Heap kısmı	Sıralı kısım
6	9, 6, 8, 2, 5, 7	
5	8, 6, 7, 2, 5	9
4	7, 6, 5, 2	8, 9
3	6, 2, 5	7, 8, 9
2	5, 2	6, 7, 8, 9
1	2	5, 6, 7, 8, 9

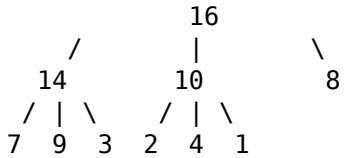
Çıktı: `2, 5, 6, 7, 8, 9`. Hoca ilerlemeyi şöyle özetledi: 9 kökte, sona taşı, tamam; 8 kökte, sona taşı, iki eleman tamam; 7, 6, 5 için aynısı; heapify edilecek bir şey kalmayınca dur.

> **Ek bilgi (derste anlatılmadı):** Heap sort ek dizi kullanmadan, girdi dizisinin içinde çalışır; yani yerinde (in-place) bir sıralamadır. Ancak kararlı (stable) değildir: Eşit anahtarlı iki elemanın girdideki göreli sırası çıktıda korunmayabilir. Bu yönüyle, derste daha önce görülen eklemeli sıralamadan ayrılır.

15. Üçlü Heap ve d'li Heap

Hoca dersin sonunda heap'in temel özelliklerini yeniden sıraladı: Max heap'te en büyük eleman, min heap'te en küçük eleman hep köktedir; ebeveynler çocuklardan her zaman büyük (max heap) ya da küçük (min heap) olur; ağaç mutlaka complete binary tree olmalıdır.

Ardından üçlü heap (ternary heap) olup olamayacağını sordu: Olur. Üçlü heap'te her ebeveynin üç çocuğu olur. Hoca `16, 14, 10, 8, 7, 9, 3, 2, 4, 1` verisini üçlü heap olarak şöyle yerleştirdi:



- 16; 14, 10 ve 8'den büyük.
- 14; 7, 9 ve 3'ten büyük.
- 10; 2, 4 ve 1'den büyük.

Her düğümün altında, çocuk varsa üçer çocuk bulunur; bu bir üçlü heap'tir. `MAX-HEAPIFY`, insert ve extract max, düğümün bütün çocuklarıyla karşılaştırma yapılacak şekilde güncellenerek kullanılabilir.

Peki üçlü heap bir şey kazandırır mı? Yükseklik $\log_3 n$ olur. Ama her seviyede karşılaştırılacak çocuk sayısı da arttığı için toplam karşılaştırma sayısında temelde bir kazanç olmaz. İstenirse n'li (genel olarak d'li) heap yapılabilir, ama en çok ikili heap kullanılır, çünkü ikili ağacın özelliklerinden yararlanır.

> **Ek bilgi (derste anlatılmadı):** 1 tabanlı dizide d'li heap için indisler genelleştirilebilir: i. düğümün çocukları $d(i-1)+2, \ldots, di+1$ gözlerinde, ebeveyni $\lfloor (i-2)/d \rfloor + 1$ gözündedir. $d = 3$ için 1. düğümün çocukları 2, 3, 4; 2. düğümünkiler 5, 6, 7; 3. düğümünkiler 8, 9, 10 olur. Bu, yukarıdaki üçlü heap çizimindeki yerleşimle aynıdır.

16. Algoritmaların C Karşılığı

> **Ek bilgi (derste anlatılmadı):** Derste algoritmalar sözde kod (pseudocode) olarak verildi. Aşağıdaki C kodu, derste anlatılan `MAX-HEAPIFY`, `BUILD-MAX-HEAP`, extract max, increase key, insert ve heap sort algoritmalarının bu nota eklenmiş karşılığıdır. Dizi derste olduğu gibi 1 tabanlıdır ve $-\infty$ yerine `INT_MIN` kullanılır. Kod `gcc -Wall` ile uyarısız derlendi ve çıktıları derste çözülen örneklerle aynı çıktı.

```c
#include <stdio.h>
#include <limits.h>
```

```c
#define MAX 100

/* Dizi 1 tabanlıdır: A[0] kullanılmaz, heap A[1..n] aralığındadır. */
#define LEFT(i)   (2 * (i))
#define RIGHT(i)  (2 * (i) + 1)
#define PARENT(i) ((i) / 2)

void swap(int A[], int i, int j) {
    int t = A[i];
    A[i] = A[j];
    A[j] = t;
}

void max_heapify(int A[], int i, int n) {
    int l = LEFT(i), r = RIGHT(i), largest = i;
    if (l <= n && A[l] > A[largest]) largest = l;
    if (r <= n && A[r] > A[largest]) largest = r;
    if (largest != i) {
        swap(A, i, largest);
        max_heapify(A, largest, n);
    }
}

void build_max_heap(int A[], int n) {
    for (int i = n / 2; i >= 1; i--)
        max_heapify(A, i, n);
}

/* Heap boşsa -1 döndürür; değilse kökü döndürür ve *n'i bir azaltır. */
int heap_extract_max(int A[], int *n) {
    if (*n < 1) return -1;
    int max = A[1];
    A[1] = A[*n];
    (*n)--;
    max_heapify(A, 1, *n);
    return max;
}

void heap_increase_key(int A[], int i, int key) {
    if (key < A[i]) return;
    A[i] = key;
    while (i > 1 && A[PARENT(i)] < A[i]) {
        swap(A, i, PARENT(i));
        i = PARENT(i);
    }
}

void max_heap_insert(int A[], int *n, int key) {
    (*n)++;
    A[*n] = INT_MIN;          /* eksi sonsuz */
    heap_increase_key(A, *n, key);
}

void heapsort(int A[], int n) {
    build_max_heap(A, n);
    for (int i = n; i >= 2; i--) {
        swap(A, 1, i);
        n--;
        max_heapify(A, 1, n);
    }
```

```
}

void print(const char *s, int A[], int n) {
    printf("%s:", s);
    for (int i = 1; i <= n; i++) printf(" %d", A[i]);
    printf("\n");
}

int main(void) {
    int A[MAX] = {0, 4, 1, 3, 2, 16, 9, 10, 14, 8, 7};
    int n = 10;
    build_max_heap(A, n);
    print("build", A, n);

    heap_increase_key(A, 9, 15);
    print("increase", A, n);
    max_heap_insert(A, &n, 12);
    max_heap_insert(A, &n, 20);
    print("insert", A, n);

    int B[MAX] = {0, 20, 15, 8, 10, 5, 7, 6, 2, 9, 1};
    int m = 10;
    int x = heap_extract_max(B, &m);
    printf("extract %d ->", x);
    print("", B, m);

    int C[MAX] = {0, 2, 9, 7, 6, 5, 8};
    heapsort(C, 6);
    print("heapsort", C, 6);
    return 0;
}
```

Çıktı:

```
build: 16 14 10 8 7 9 3 2 4 1
increase: 16 15 10 14 7 9 3 2 8 1
insert: 20 15 16 14 12 10 3 2 8 1 7 9
extract 20 ->: 15 10 8 9 5 7 6 2 1
heapsort: 2 5 6 7 8 9
```

Örnek Sorular

Soru 1 (derste sözlü, 00:15:43)

Aşağıdaki üç ikili ağaç (dizi gösterimiyle) heap midir? `10, 5, 7, 4, 2, 1`; `10, 5, 7, _, 2, 1` (4. göz boş); `10, 5, 7, 6, 2, 1`.

Çözüm (derste verildi):

Dizi	Complete binary tree mi?	Heap koşulu?	Sonuç
10, 5, 7, 4, 2, 1	Evet	Evet	Heap
10, 5, 7, _, 2, 1	Hayır, son seviyede soldan boşluk var	Bakmaya gerek yok	Heap değil
10, 5, 7, 6, 2, 1	Evet	Hayır, $5 < 6$	Heap değil

Soru 2 (derste sözlü, 00:28:37)

3 elemanlı bir complete ikili ağacın yüksekliği kaçtır? $\log_2 3$ kaç eder?

Çözüm (derste verildi): $\log_2 3 \approx 1.58$. 4 elemanlı ağacın yüksekliği 2'dir ($\log_2 4 = 2$), 8 elemanda yükseklik 3 olur.

> **Çözüm (derste verilmedi):** Hocanın "yuvarlayınca 2" ifadesi yanlıştır. Yükseklik $\lfloor \log_2 n \rfloor$'dir; 3 elemanlı ağacın yüksekliği $\lfloor 1.58 \rfloor = 1$'dir. Python ile: $n = 3 \to 1$, $n = 4 \to 2$, $n = 7 \to 2$, $n = 8 \to 3$, $n = 10 \to 3$.

Soru 3 (derste sözlü, 01:20:03)

`16, 15, 10, 14, 7, 9, 3, 2, 8, 1` max heap'ine yeni bir eleman nasıl eklenir? Önce 12, sonra 20 eklenince heap ne olur?

Çözüm (derste verildi): Heap boyutu bir artırılır, yeni son göze $-\infty$ yazılır (heap bozulmaz), sonra `HEAP-INCREASE-KEY` ile değer eklenecek anahtar yapılır.

- 12: 7 ile yer değiştirir, 15'in altında durur. `16, 15, 10, 14, 12, 9, 3, 2, 8, 1, 7`.
- 20: sırayla 9, 10 ve 16 ile yer değiştirip köke çıkar. `20, 15, 16, 14, 12, 10, 3, 2, 8, 1, 7, 9`.

Soru 4 (derste sözlü, 01:27:44)

Bottom-up ve top-down yaklaşımlarla aynı diziden oluşturulan heap'ler aynı olur mu?

Çözüm (derste verildi): Olmak zorunda değildir. `1, 2, 3` girdisi için bottom-up `3, 2, 1`, top-down `3, 1, 2` verir. İkisi de geçerli max heap'tir.

Dersin Toparlaması

Bu derste heap veri yapısı tanıtıldı. Heap, complete binary tree olan ve her ebeveynin çocuklarından büyük (max heap) ya da küçük (min heap) olduğu bir ağaçtır. İkili arama ağacının sol-sağ düzenini taşımaz; tek garantisi en büyük ya da en küçük elemanın kökte olmasıdır. Dizi gösteriminde kök $A[1]$'dedir, çocuklar $2i$ ve $2i+1$'de, ebeveyn $\lfloor i/2 \rfloor$'dedir; çocuğu olan düğümler ilk $\lfloor n/2 \rfloor$ gözdedir. Ağacın yüksekliği $\log n$ mertebesinde olduğu için `MAX-HEAPIFY`, extract max, increase key ve insert işlemleri en kötü durumda $O(\log n)$, en büyüğü okumak $O(1)$'dir.

Heap iki yolla oluşturulabilir: $n/2$'den köke doğru `MAX-HEAPIFY` çağıran bottom-up yöntem ve elemanları tek tek insert eden top-down yöntem. İki yöntem aynı heap'i vermek zorunda değildir. Son olarak heap sort görüldü: Önce heap kurulur, sonra kök tekrar tekrar sona atılıp heap küçültülür ve yeniden heapify edilir; toplam karmaşıklık $O(n \log n)$'dir. Hoca heap sort'u "dönüştür ve fethet" yaklaşımının en temiz örneklerinden biri olarak gösterdi ve dersi üçlü ile d'li heap'lerin mümkün olduğunu, ama pratikte en çok ikili heap'in kullanıldığını belirterek kapattı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.